

PROGRAMA DE FORMACIÓN Y ACOMPAÑAMIENTO QA: JAVA, TESTING Y DEVOPS

Curso completo de capacitación técnica avanzada con soporte para el equipo de QA

Tabla de Contenidos

- 1.0 Cronograma de Clases
- 2.0 Estructura del Programa
- 3.0 Acompañamiento QA
- 4.0 Presupuesto y Modelos de Precio
- 5.0 Condiciones Comerciales
- 6.0 Consultoría de Despliegues y Costos GitHub (Meses 1–6)

1.0 CRONOGRAMA COMPLETO DE APRENDIZAJE JAVA, TESTING Y DEVOPS

MÓDULO 1 — Java de 0 a 100 (Aprox 60 clases)

BLOQUE 1: Fundamentos (Clases 1-10)

Clase	Tema
1	Introducción a Java: historia, JDK vs JRE vs JVM, instalación, primer Hello World
2	Variables, tipos de datos primitivos (int , double , boolean , char , String)
3	Operadores aritméticos, relacionales y lógicos
4	Estructuras de control: if , else if , else , operador ternario
5	Bucles: for , while , do-while

Clase	Tema
6	switch / switch expressions (Java 14+)
7	Arrays unidimensionales y multidimensionales
8	Métodos: definición, parámetros, retorno, sobrecarga
9	Recursividad: factorial, Fibonacci, torres de Hanói
10	Manejo de entrada/salida: Scanner , System.out

BLOQUE 2: Programación Orientada a Objetos (Clases 11-25)

Clase	Tema
11	Clases y objetos: atributos, métodos, constructores
12	Encapsulamiento: private , public , protected , getters/setters
13	Herencia: extends , uso de super
14	Polimorfismo: sobreescritura (@Override), casting
15	Clases abstractas vs interfaces
16	Interfaces: implementación múltiple, default methods
17	Modificadores: static , final , this
18	Clases internas, anónimas y locales
19	Enumeraciones (enum) y anotaciones básicas
20	Paquetes, imports y gestión de dependencias con Maven
21	Manejo de excepciones: try-catch-finally , throws
22	Excepciones personalizadas, jerarquía de excepciones
23	Proyecto práctico OOP: Sistema de Biblioteca
24	UML básico: diagramas de clase aplicados a Java
25	Revisión y ejercicios integradores de OOP

BLOQUE 3: Colecciones y Genéricos (Clases 26-33)

Clase	Tema
26	ArrayList , LinkedList , diferencias y cuándo usar cada una
27	HashMap , HashSet , LinkedHashMap
28	TreeMap , TreeSet , ordenación natural
29	Iteradores, for-each , Iterator
30	Genéricos: <T> , wildcards <?> , bounded types
31	Collections utility class: sort , shuffle , binarySearch
32	Stack , Queue , Deque , PriorityQueue
33	Proyecto práctico: Gestión de inventario con colecciones

BLOQUE 4: Java Funcional y Streams (Clases 34-40)

Clase	Tema
34	Expresiones lambda: sintaxis, contexto funcional
35	Interfaces funcionales: Predicate , Function , Consumer , Supplier
36	Stream API: filter , map , collect , reduce
37	Streams avanzados: flatMap , groupingBy , partitioningBy
38	Optional<T> : evitar NullPointerException
39	Referencia a métodos (Method References)
40	Proyecto práctico: Procesamiento de datos con Streams

BLOQUE 5: I/O, Serialización y Concurrencia (Clases 41-48)

Clase	Tema
41	Manejo de archivos: File , Path , Files (NIO.2)
42	Lectura/escritura: BufferedReader , BufferedWriter , FileInputStream
43	Serialización y deserialización de objetos
44	Manejo de JSON con Jackson o Gson

Clase	Tema
45	Hilos (Thread): creación, ciclo de vida
46	Runnable , Callable , ExecutorService , Future
47	Sincronización: synchronized , volatile , ReentrantLock
48	CompletableFuture y programación asíncrona básica

BLOQUE 6: Bases de Datos y APIs (Clases 49-56)

Clase	Tema
49	JDBC: conexión a BD, DriverManager , Connection
50	CRUD con JDBC: PreparedStatement , ResultSet
51	Introducción a JPA / Hibernate: entidades, EntityManager
52	Relaciones JPA: @OneToMany , @ManyToMany , @JoinColumn
53	Spring Boot: configuración inicial, estructura del proyecto
54	REST API con Spring Boot: @RestController , @GetMapping , @PostMapping
55	Spring Data JPA: repositorios, consultas JPQL
56	Proyecto práctico: API REST CRUD completa con Spring Boot

BLOQUE 7: Temas Avanzados (Clases 57-60)

Clase	Tema
57	Patrones de diseño: Singleton, Factory, Builder, Observer
58	Principios SOLID aplicados a Java
59	Herramientas del ecosistema: Maven, Git, Docker básico
60	Proyecto final: aplicación completa con Spring Boot, JPA, REST y manejo de errores

MÓDULO 2 — Pruebas Unitarias con JUnit (Aprox 20 clases)

Clase	Tema
61	Qué es una prueba unitaria, principios F.I.R.S.T, pirámide de testing
62	Configuración de JUnit 5 con Maven/Gradle
63	Anatomía de un test: <code>@Test</code> , <code>@BeforeEach</code> , <code>@AfterEach</code> , <code>@BeforeAll</code>
64	Assertions: <code>assertEquals</code> , <code>assertTrue</code> , <code>assertThrows</code> , <code>assertAll</code>
65	Ciclo de vida de tests y anotaciones de ciclo (<code>@Nested</code>)
66	Tests parametrizados: <code>@ParameterizedTest</code> , <code>@ValueSource</code> , <code>@CsvSource</code>
67	<code>@MethodSource</code> , <code>@EnumSource</code> y fuentes de datos externas
68	Mocking con Mockito: <code>@Mock</code> , <code>@InjectMocks</code> , <code>when().thenReturn()</code>
69	Verificación con Mockito: <code>verify()</code> , <code>times()</code> , <code>ArgumentCaptor</code>
70	Spies, stubs y mocks: diferencias y cuándo usarlos
71	TDD (Test Driven Development): ciclo Red-Green-Refactor
72	Cobertura de código con JaCoCo
73	Testing de excepciones y casos negativos
74	Testing de clases con dependencias: inyección con <code>@ExtendWith</code>
75	Testing de repositorios con H2 en memoria
76	Testing de servicios Spring Boot con <code>@SpringBootTest</code>
77	<code>@WebMvcTest</code> y <code>MockMvc</code> : testing de controladores REST
78	Testing de componentes aislados con <code>@DataJpaTest</code>
79	Organización de suites de tests, tags (<code>@Tag</code>), exclusiones
80	Buenas prácticas: nombres de tests, AAA pattern, mantenibilidad

MÓDULO 3 — Pruebas Unitarias con JUnit para Programas AS400 / IBM i (Aprox 15 clases)

Clase	Tema
81	Introducción al entorno IBM i / AS400: estructura, objetos, RPGLE
82	Arquitectura de integración Java ↔ AS400: JT400 / IBM Toolbox for Java
83	Configuración de dependencia <code>jt400.jar</code> en Maven
84	Conexión a AS400 con <code>AS400</code> class, <code>SecureAS400</code> , autenticación
85	Llamada a programas RPG con <code>ProgramCall</code> y <code>ProgramParameter</code>
86	Llamada a comandos CL con <code>CommandCall</code>
87	Estructuras de datos (Data Structures) y mapeo Java ↔ AS400
88	Lectura de archivos físicos con <code>KeyedFile</code> , <code>SequentialFile</code>
89	Acceso a colas de datos (<code>DataQueue</code>) y colas de mensajes
90	Estrategia de testing: mockear JT400 con Mockito
91	Mock de <code>AS400</code> , <code>ProgramCall</code> , <code>ProgramParameter</code> en tests unitarios
92	Testing de servicios que llaman programas RPG con datos de prueba
93	Testing de lectura/escritura de archivos físicos (mocked)
94	Integración con JUnit 5: suites de tests para módulos AS400
95	Proyecto práctico: suite de tests unitarios para módulo AS400 completo

MÓDULO 4 — Pruebas de Aceptación con Karate (15 clases)

Clase	Tema
96	Qué es Karate: BDD + API testing, comparación con RestAssured
97	Configuración del proyecto Karate con Maven
98	Estructura de un feature file: <code>Feature</code> , <code>Scenario</code> , <code>Given/When/Then</code>
99	Primeras pruebas HTTP: <code>GET</code> , <code>POST</code> , validación de <code>status</code> y <code>response</code>
100	Sintaxis Karate: variables, <code>def</code> , <code>match</code> , <code>contains</code> , <code>==</code>
101	JSON y XML: matching parcial, <code>#notnull</code> , <code>#array</code> , esquemas

Clase	Tema
102	PUT , PATCH , DELETE : pruebas de API REST completa
103	Parametrización con Examples: en Scenario Outline
104	Llamadas entre features: call , callonce , reutilización
105	Autenticación: Basic Auth, Bearer Token, OAuth2
106	Headers, cookies, multipart y subida de archivos
107	Configuración de entornos: karate-config.js , perfiles
108	Mocking de servicios con Karate Netty (karate-mock)
109	Reportes HTML con Karate Reports y integración con CI/CD
110	Proyecto práctico: suite de integración completa para una API REST

MÓDULO 5 — Pruebas E2E con Playwright + Serenity BDD (Aprox 20 clases)

Clase	Tema
111	Introducción a E2E testing: propósito, cuándo usarlo, herramientas
112	Playwright: instalación, arquitectura, soporte multi-browser
113	Primer test E2E con Playwright en Java (playwright-java)
114	Selectores: CSS, XPath, getByRole , getByText , getByLabel
115	Acciones: click , fill , select , hover , press , dragAndDrop
116	Assertions con expect() : toBeVisible , toHaveText , toHaveURL
117	Waits inteligentes: auto-wait de Playwright, waitForSelector , timeouts
118	Page Object Model (POM) con Playwright
119	Manejo de ventanas, popups, iframes y file downloads
120	Interceptación de red: route() , mock de APIs en E2E
121	Introducción a Serenity BDD: filosofía, reportes vivos

Clase	Tema
122	Configuración de Serenity con JUnit 5 y Maven
123	@Steps y StepLibrary : pasos reutilizables con Serenity
124	Integración Playwright + Serenity: SerenityRunner , anotaciones
125	Escritura de tests BDD con Cucumber + Serenity: feature files
126	@Given , @When , @Then en Step Definitions con Serenity
127	Screenshots automáticos, narrativas y reportes Serenity HTML
128	Datos de prueba: tablas en Cucumber, Faker, fixtures
129	Paralelización de tests E2E y configuración de CI/CD (GitHub Actions)
130	Proyecto final: suite E2E completa con Playwright + Serenity + BDD

NOTA: Cada módulo construye sobre el anterior. Se debe incluir todo un esquema de practica, talleres, mini proyectos, se puede completar en un aproximado de **18 meses**.

MÓDULO 6 — DevOps Básico + GitHub Actions + Automatización de Pruebas (30 clases)

BLOQUE 1: Fundamentos de DevOps (Clases 131–136)

Clase	Tema
131	Qué es DevOps: cultura, principios, ciclo CI/CD, diferencia Dev vs Ops
132	Control de versiones con Git: clone , commit , branch , merge , rebase
133	Flujos de trabajo Git: GitFlow, trunk-based development, pull requests
134	Introducción a contenedores: qué es Docker, imágenes vs contenedores
135	Docker práctico: Dockerfile , docker build , docker run , docker-compose
136	Registros de imágenes: Docker Hub, GitHub Container Registry (ghcr.io)

BLOQUE 2: GitHub Actions desde cero (Clases 137–144)

Clase	Tema
137	Qué es GitHub Actions: eventos, workflows, jobs, steps
138	Anatomía de un workflow YAML: on , jobs , runs-on , steps , uses
139	Eventos de disparo: push , pull_request , schedule , workflow_dispatch
140	Actions del marketplace: actions/checkout , actions/setup-java , actions/cache
141	Variables de entorno, env , secretos con secrets.* y vars.*
142	Jobs en paralelo y secuenciales: needs , matrices con strategy.matrix
143	Artefactos: upload-artifact , download-artifact , retención
144	Reusable workflows: workflow_call , reutilización entre repositorios

BLOQUE 3: Pipeline de Build y Calidad de Código (Clases 145–148)

Clase	Tema
145	Pipeline de compilación Java con Maven: mvn compile , package , verify
146	Análisis estático de código: Checkstyle, SpotBugs, PMD en pipeline
147	Integración con SonarCloud: escaneo de calidad automático en PR
148	Caché de dependencias Maven/Gradle en GitHub Actions para acelerar builds

BLOQUE 4: Ejecución de Pruebas Unitarias en Pipeline (Clases 149–151)

Clase	Tema
149	Job de JUnit 5 en GitHub Actions: mvn test , reporte de resultados
150	Publicar resultados de tests con dorny/test-reporter y junit-reporter
151	Cobertura de código con JaCoCo: generar reporte y publicar en PR como comentario

BLOQUE 5: Ejecución de Pruebas de Integración con Karate (Clases 152–153)

Clase	Tema
152	Levantar la API bajo prueba con Docker en el pipeline: <code>services</code> y <code>docker-compose</code>
153	Job de Karate en GitHub Actions: configurar entorno, ejecutar tests, publicar reporte HTML como artefacto

BLOQUE 6: Ejecución de Pruebas E2E con Playwright + Serenity (Clases 154–156)

Clase	Tema
154	Instalación de browsers en CI: <code>playwright install --with-deps</code> en pipeline
155	Job E2E headless con Playwright + JUnit en GitHub Actions
156	Publicar reporte Serenity HTML como artefacto y como GitHub Pages

BLOQUE 7: Pipeline Completo y Buenas Prácticas (Clases 157–160)

Clase	Tema
157	Pipeline multi-stage completo: build → unit tests → integration tests → E2E
158	Estrategia de ramas: bloquear merge si algún stage falla, branch protection rules
159	Notificaciones: Slack, email y comentarios automáticos en PR con resultados
160	Proyecto final: pipeline end-to-end desde commit hasta reporte publicado

NOTA: Cada módulo construye sobre el anterior. Se debe incluir todo un esquema de practica, talleres, mini proyectos, se puede completar en un aproximado de **18 meses**.

2.0 Estructura del Programa

Distribución de Módulos

Módulo	Tema	Clases	Duración
1	Java de 0 a 100	60	~60 horas

Módulo	Tema	Clases	Duración
2	Pruebas Unitarias con JUnit	20	~20 horas
3	JUnit para AS400/IBM i	15	~15 horas
4	Pruebas de Integración con Karate	15	~15 horas
5	Pruebas E2E con Playwright + Serenity	20	~20 horas
6	DevOps Básico + GitHub Actions	30	~30 horas
TOTAL		160	~160 horas

Tiempo estimado del programa académico: 18 meses con esquema completo de prácticas, talleres y mini proyectos.

Metodología de Enseñanza

Ciclo de cada clase (Sesion en vivo 1 hora):

1. **Conceptos (20 min):** Explicación, fundamentos de programación
2. **Live Coding (30 min):** Escribimos código juntos, QA
3. **Entrega de Taller (10 min):** Explicacion del taller a realizar

Ciclo de cada clase (Taller y dudas 1 hora):

1. **Revision de taller (30 min):** Corrección de la construccion del taller
2. **Respuesta a dudas (20 min):** Respues a dudas tecnicas y funcionales
3. **Introduccion siguiente tema (10 min):** Revision de la teoria y practica del siguiente tema

Principios:

- **Hands-on:** 70% código, 30% teoría
- **Iterativo:** Construimos sobre lo anterior
- **Asincrónico-friendly:** Grabaciones disponibles
- **Dudas:** A un chat de distancia

3.0 Acompañamiento QA

Adicionalmente al programa de clases, ofreceré acompañamiento al equipo de QA durante **12 meses**, iniciando **después de los primeros 6 meses de clases**.

Cupo máximo aplicable a esta propuesta: hasta **20 personas** tanto para las clases como para el acompañamiento.

Modelo de trabajo en paralelo: el acompañamiento al área de QA en DevOps, pipelines y ejecución de pruebas automatizadas puede ejecutarse en paralelo con el acompañamiento al equipo de desarrollo de software para la configuración de pipelines y despliegues automatizados de las líneas de producto.

Alcance del acompañamiento

- Apoyo en la creación de artefactos de pruebas automatizadas para los productos de la empresa.
- Definición y estandarización de buenas prácticas para pruebas unitarias, de integración y E2E.
- Acompañamiento en la construcción de frameworks, suites, datos de prueba y reportes.
- Revisión técnica de automatizaciones y apoyo para su integración en CI/CD.
- Mentoría al equipo de QA para fortalecer autonomía operativa.

Entregables sugeridos

- Matrices de pruebas y trazabilidad de cobertura.
- Repositorios o módulos de automatización organizados por producto o flujo.
- Plantillas reutilizables para scripts, fixtures y reportes.
- Recomendaciones de mantenimiento y escalabilidad del ecosistema de pruebas.

¿Por qué elegir este programa vs. otras opciones?

Aspecto	Bootcamp Online	Consultoría Externa	Tu Propuesta
Sincrónico	✗ Grabado	✓ Sí	✓ Sí (real-time)
Especialización Java/AS400	✗ Genérico	✓ Sí	✓ Sí (diferencial)
Aplicación en contexto real	✗ Ejercicios ficticios	⚠ Parcial	✓ Tus productos
Acompañamiento post-formación	✗ No	✓ Sí (costoso)	✓ Incluido

Aspecto	Bootcamp Online	Consultoría Externa	Tu Propuesta
Garantía de autonomía del equipo	✗ No	⚠ Parcial	✅ Explícita
Precio/valor 18 meses	COP \$600k/mes	COP \$5M-8M/mes	COP \$2.4M → \$4M

Ventaja competitiva

- Sincrónico (real-time mentoring vs. pregrabado).
- Especialización Java + AS400 + QA.
- Acompañamiento en contexto real (sus productos).
- Garantía: equipo autónomo en automatización al finalizar.

Modalidad sugerida

- Inicio: mes 7 del proyecto académico (después de los primeros 6 meses de clases).
- Duración: 12 meses de acompañamiento, con posibilidad de extensión.
- Frecuencia: espacios semanales de seguimiento técnico para diseño, construcción y estabilización de componentes de pruebas automatizadas.
- Modalidad de cobro: incluida dentro de la mensualidad de COP \$4,000,000 durante la etapa de meses 7 a 18.



Hitos de Entrega y Éxito

Al finalizar mes 18:

- **Cobertura de código:** ≥ 70% cobertura unitaria en módulos críticos
- **Suites automatizadas:** ≥ 3 suites funcionales (JUnit + Karate + E2E)
- **Pipeline CI/CD:** Automatización de pruebas en GitHub Actions
- **Documentación:** Guías de mantenimiento y escalabilidad
- **Autonomía:** Equipo QA capaz de crear nuevas suites sin asistencia

Indicadores de progreso:

- Mes 6: Fundamentos Java sólidos + primeras suites unitarias
- Mes 12: Karate + pipelines CI/CD configuradas

- Mes 18: Suite E2E funcional + equipo independiente

4.0 PRESUPUESTO Y MODELOS DE PRECIO

Para un programa completo de 160 clases con modelo integral (clase + seguimiento).

Análisis de Supuestos

Desglose de Clases

- Total de clases: 160
 - Módulo 1: 60 clases
 - Módulo 2: 20 clases
 - Módulo 3: 15 clases
 - Módulo 4: 15 clases
 - Módulo 5: 20 clases
 - Módulo 6: 30 clases

Modelo de Tiempo por Clase

Actividad	Horas
Material (script, slides, labs)	1 h
Sesión en vivo	1 h
Taller y Dudas	1 h
Total por clase	3 h

Horas totales del instructor: 160 clases × 3 h = **480 horas**

Valor de hora por clase: COP \$100,000

Valor Mensual: COP \$100,000 x 6 h x 4 semanas = **COP \$2,400,000**

Se dictarán 2 clases por semana (promedio) para completar el plan en 18 meses.

Modelo de Tiempo Consultoría de Despliegues Automatizados (Meses 1 a 6)

Actividad	Horas
Diseño de pipelines y estrategia por línea de producto (Dev/QA/Prod)	2 h
Implementación guiada (workflows, build, release, deploy)	1 h
Soporte y seguimiento técnico semanal	1 h
Total consultoría semanal	4 h

Valor de hora consultoría: COP \$100,000

Valor mensual consultoría (meses 1 a 6): COP \$100,000 x 4 h x 4 semanas = **COP \$1,600,000**

Valor mensual total meses 1 a 6 (clases + consultoría): COP \$2,400,000 + COP \$1,600,000 = **COP \$4,000,000**

Modelo de Tiempo Acompañamiento area de QA

Actividad	Horas
Reuniones de acompañamiento	2 h
Soporte en construccion de pruebas automatizas	2 h
Total por acompañamiento	4 h

Valor de hora por acompañamiento: COP \$100,000

Valor Mensual: COP \$100,000 x 4 h x 4 semanas = **COP \$1,600,000**

Se Acompañara al area de QA 4 horas semanales (promedio) a partir del mes 7 del programa de clases.

Total Mensual a partir del mes 7: COP \$2,400,000 + COP \$1,600,000 = **COP \$4,000,000**

 **No incluye:** infraestructura/labs/VMs, desplazamiento presencial, ni licencias

Estructura Económica Propuesta

Resumen económico por etapas

Etapas	Alcance	Valor mensual	Duración
Meses 1 a 6	Clases del programa académico + consultoría y apoyo para construir despliegues automatizados (Dev, QA y Producción) para 7 líneas de producto	COP \$4,000,000	6 meses
Meses 7 a 18	Continuación de clases + acompañamiento al área de QA para construcción de componentes de pruebas automatizadas	COP \$4,000,000	12 meses
Meses 18 a 24	Optimización y escalabilidad CI/CD (endurecimiento, mantenimiento y transferencia final)	COP \$4,000,000	6 meses

Modalidad de cobro sugerida como profesional independiente

- **Estructura contractual:** prestación de servicios profesionales.
- **Forma de cobro:** mensualidad fija por etapas, de acuerdo con el alcance definido en cada periodo.
- **Etapas 1 (meses 1 a 6):** COP \$4,000,000 mensuales por clases + consultoría y apoyo para despliegues automatizados.
- **Etapas 2 (meses 7 a 18):** COP \$4,000,000 mensuales por continuación de clases más acompañamiento al área de QA.
- **Etapas 3 (meses 18 a 24):** COP \$4,000,000 mensuales por optimización, estabilización y transferencia final de la operación CI/CD.

5.0 Condiciones Comerciales

Pagos

- **Meses 1 a 6:** COP \$4,000,000 mensuales por continuidad de clases del programa + consultoría y apoyo para despliegues automatizados en Dev, QA y Producción.
- **Meses 7 a 18:** COP \$4,000,000 mensuales por continuidad de clases más acompañamiento al área de QA.
- **Meses 18 a 24:** COP \$4,000,000 mensuales por optimización, escalabilidad y transferencia final de la operación CI/CD.
- **Plazo de pago sugerido:** A los 30 días calendario con previa presentación de la cuenta de cobro.

Alcance y exclusiones

- El valor del programa incluye sesión en vivo, preparación de material y seguimiento básico asociado a cada clase.
- El valor del acompañamiento incluye presencia en sesiones de construcción de componentes de pruebas automatizadas y apoyo técnico.
- El valor del acompañamiento Diseño de estrategia, templates, pair programming, revisión de 2-3 suites piloto, capacitación en buenas prácticas.
- El valor de la consultoría de despliegues (meses 1 a 6) incluye definición de estrategia GitHub, construcción de workflows CI/CD, compilación y despliegue automatizado hacia Dev/QA/Prod para 7 líneas de producto.
- El acompañamiento al área de QA puede ejecutarse en paralelo con el acompañamiento al equipo de desarrollo para la configuración y estabilización de pipelines y despliegues automatizados por línea de producto.
- Esta propuesta aplica para un **máximo de 20 personas** entre participantes de clases y acompañamiento. Si se supera este límite, se deberá revisar alcance y tarifa.
- No se incluye desarrollo desde 0 de frameworks ni construcción de todos los tests de los productos.
- No incluye servidores o ambientes donde se despliegan las soluciones (ASNET ya dispone de estos recursos), ni infraestructura, laboratorios, licencias, desplazamientos, viáticos o desarrollos adicionales por fuera del alcance acordado.

Reprogramaciones

- Las sesiones podrán reprogramarse con previo aviso.

Disponibilidad horaria para clases y acompañamiento

- Lunes a viernes: **7:00 a.m. a 9:00 a.m. y 5:00 p.m. a 9:00 p.m.**
- Sábados: **disponibilidad de jornada completa**
- La asignación de espacios específicos se coordinará semanalmente con el equipo de ASNET.

Supuestos Críticos del Programa

Para que el programa tenga éxito, se requiere:

Del lado de ASNET:

- Equipo QA dedicado $\geq 50\%$ del tiempo al programa

- Acceso a ambientes de prueba estables (dev/staging)
- Participación de stakeholders en diseño de suite de tests
- Compromiso de aplicar aprendizajes en contexto real

Del lado del instructor:

- Disponibilidad de lunes a viernes de 7:00 a.m. a 9:00 a.m. y de 5:00 p.m. a 9:00 p.m., y sábados en jornada completa
- Respuesta a dudas máximo 12h hábiles
- Acceso a repositorio de la empresa para pair programming
- Acceso a ambiente de QA para revision de los componentes de pruebas automatizadas

Riesgos y contingencias:

Riesgo	Probabilidad	Impacto	Mitigación
Equipo no disponible $\geq 50\%$	Media	Alto	Reprogramar clases, extensión del cronograma
Ambientes inestables	Alta	Medio	Usar Docker/mockups, labs virtuales
Rotación de personal QA	Media	Alto	Documentación acelerada, grabaciones
Baja adopción de automatización	Baja	Medio	Sesiones extra de mentoría, pair programming

6.0 CONSULTORÍA DE DESPLIEGUES AUTOMATIZADOS Y COSTOS GITHUB

Implementación inicial: Meses 1 a 6 (paralela a las clases)

Esta consultoría se ejecuta en paralelo con las clases del programa durante los primeros 6 meses. Si se aprueba esta etapa, los honorarios mensuales del instructor serán de **COP \$4,000,000**, incluyendo clases + apoyo/consultoría para construir despliegues automatizados.

Durante esta etapa, el acompañamiento de QA (en automatización de pruebas, DevOps y operación de pipelines) se coordina en paralelo con el acompañamiento al equipo de desarrollo

para la configuración, adopción y madurez de los pipelines de despliegue automatizado en las líneas de producto.

Equipo QA: 8 personas
Líneas de Producto: 7 (AS400, APIs, Web, Móvil)
Ambiente: QA Existente (sin infraestructura adicional)
Tasa de Cambio de referencia: USD 1 = COP \$3,500

Resumen Ejecutivo de Costos de Infraestructura

Estimado Mensual Total (pruebas + despliegues, sin incluir servidores): USD \$321 - \$515 / mes
(COP \$1,123,500 - \$1,802,500)

Componente	Costo Mensual USD	Costo Mensual COP
GitHub Team (8 usuarios)	USD \$168	COP \$588,000
GitHub Actions — Pruebas automatizadas (~5,400 min)	USD \$19 - \$80	COP \$66,500 - \$280,000
GitHub Actions — Despliegues 7 líneas (~6,710 min)	USD \$55 - \$120	COP \$192,500 - \$420,000
Soporte/Incidencias	USD \$50 - \$100	COP \$175,000 - \$350,000
Contingencia (10%)	USD \$29 - \$47	COP \$101,500 - \$164,500
TOTAL MENSUAL	USD \$321 - \$515	COP \$1,123,500 - \$1,802,500

 Este costo de infraestructura es **independiente y adicional** a los honorarios del instructor. Es un costo operativo directo de ASNET. Con runners **self-hosted** (servidores existentes de ASNET), el costo se reduce a ~USD \$218 - \$268/mes (solo GitHub Team + soporte + contingencia).

Contexto y Alcance

Lo que YA TIENEN (No se requiere presupuestar):

-  Ambiente QA con máquinas de prueba
-  Bases de datos de prueba
-  Acceso a ambientes AS400, APIs, Web, Móvil
-  Infraestructura de servidores local/on-premise

Lo que SE REQUIERE (Presupuesto):

-  GitHub (Repositorio centralizado)
-  GitHub Actions (CI/CD para ejecutar pruebas)
-  Runners (GitHub-hosted o self-hosted sobre infraestructura existente)
-  Herramientas de reportes y notificaciones
-  Soporte técnico

 Desglose Detallado de Costos de Infraestructura

 1 GitHub Team (Repositorio)

Item	Costo Unitario	Cantidad	Costo Total USD	Costo Total COP
GitHub Team	USD \$21/usuario/mes	8	USD \$168/mes	COP \$588,000/mes

 **Recomendación:** Comenzar con **GitHub Team** (USD \$168/mes = COP \$588,000). Escalar a Enterprise solo si requieren integración SAML/SSO.

 2 GitHub Actions (CI/CD)

Estimación de Consumo por Línea de Producto

Línea	# Tests	Tiempo/Ejecución	Frecuencia Diaria	Min/Mes	Observaciones
AS400 (JUnit RPG)	150	30 min	2x	1,200	Pruebas contra AS400 existente
APIs (Karate)	200	15 min	4x	1,800	Tests REST a APIs existentes
Web (Playwright	300	45 min	1x	900	E2E a aplicación web

Línea	# Tests	Tiempo/Ejecución	Frecuencia Diaria	Min/Mes	Observaciones
E2E)					
Móvil (E2E)	200	30 min	1x	600	E2E a aplicación móvil
Integrales (Suite completa)	-	120 min	1x/día	900	Smoke tests nightly
TOTAL MENSUAL				~5,400 min	

- Minutos incluidos en GitHub Team: 3,000/mes
- Minutos excedentes: 2,400 min × USD \$0.008 = **USD \$19.2/mes**
- Referencia costo base GitHub-hosted (Linux): ~USD \$20/mes por excedente estimado
- Escenario con mayor paralelismo y variación de cargas: USD \$80 - \$120/mes
- Escenario self-hosted (usando servidores ya existentes de ASNET): USD \$0 por minutos/runners en GitHub, pagando solo la capa GitHub Team y operación

Validación de costos: para cargas Linux el excedente proyectado (2,400 min) es bajo y se aproxima a USD \$19.2/mes; el rango alto contempla picos de ejecución, reprocesos y paralelismo.

3

Herramientas Complementarias

Herramienta	Propósito	Costo USD/mes
Slack Integration	Notificaciones de test results	USD \$0
SonarCloud	Análisis estático Java (free tier OSS)	USD \$0
Codecov	Cobertura de código	USD \$0
GitHub Container Registry (ghcr.io)	Docker images para builds y despliegues (~300-500 MB)	USD \$0
GitHub Pages	Reportes HTML públicos	USD \$0

4

Despliegues Automatizados — Build + Deploy por Línea de Producto

Estimación de minutos de GitHub Actions consumidos exclusivamente por los pipelines de **compilación y despliegue** (no incluye pruebas, que se contabilizan por separado). Aplica para ambientes Dev, QA y Producción.

Supuestos del modelo de despliegue

- **Dev:** despliegue automático en cada push a rama `develop` (~5x por día hábil)
- **QA:** despliegue automático al fusionar a rama `release` (~2x por día hábil)
- **Prod:** despliegue con aprobación GitHub Environment (~3x por semana)
- **Días hábiles al mes:** 22

Línea de Producto	Build (min)	Deploys Dev/día	Deploys QA/día	Deploys Prod/semana	Min/mes estimados
AS400 / IBM i	10 min	3x	1x	2x	~880
API REST 1	8 min	5x	2x	3x	~1,100
API REST 2	8 min	5x	2x	3x	~1,100
API REST 3	8 min	3x	1x	2x	~770
Web (Frontend)	12 min	4x	2x	3x	~1,320
Móvil	15 min	2x	1x	2x	~880
Integraciones / Batch	10 min	2x	1x	2x	~660
TOTAL MENSUAL DESPLIEGUES					~6,710 min

Consolidado Total de Minutos (Pruebas + Despliegues)

Concepto	Min/mes
Pruebas automatizadas (JUnit, Karate, Playwright, E2E)	~5,400
Despliegues automatizados (Build + Deploy 7 líneas)	~6,710
TOTAL MENSUAL COMBINADO	~12,110 min

- **Minutos incluidos en GitHub Team:** 3,000/mes
- **Minutos excedentes:** ~9,110 min × USD \$0.008 = **~USD \$73/mes** (Linux runners)

- **Escenario pico (reprocesos, hotfixes, rollbacks):** hasta USD \$120 - \$150/mes

Validación: el excedente proyectado para Linux es conservador (~USD \$73/mes base). El rango alto contempla pipelines fallidos con retry, builds paralelos y releases urgentes a producción.

5

Herramientas Complementarias de Despliegue

Herramienta	Propósito	Costo USD/mes
GitHub Environments	Aprobaciones y protección de deploys a Producción	USD \$0 (incluido en Team)
GitHub Container Registry	Imágenes Docker versionadas por release	USD \$0
GitHub Releases	Gestión de versiones y artefactos	USD \$0
Slack / Email	Notificaciones de despliegue exitoso/fallido	USD \$0



Desglose Consolidado Recomendado (Pruebas + Despliegues)

Componente	USD/mes	COP/mes
GitHub Team (8 usuarios)	USD \$168	COP \$588,000
GitHub Actions — Pruebas (~5,400 min)	USD \$19 - \$80	COP \$66,500 - \$280,000
GitHub Actions — Despliegues 7 líneas (~6,710 min)	USD \$55 - \$120	COP \$192,500 - \$420,000
Soporte técnico	USD \$50 - \$100	COP \$175,000 - \$350,000
Contingencia (10%)	USD \$29 - \$47	COP \$101,500 - \$164,500
TOTAL MENSUAL	USD \$321 - \$515	COP \$1,123,500 - \$1,802,500

 El escenario **self-hosted** (usando servidores ya disponibles en ASNET) reduce el costo de Actions a USD \$0 en minutos, dejando solo USD \$168/mes por GitHub Team más soporte. Esta opción se evalúa en la FASE 1 de implementación.

17 Cronograma de Implementación Inicial (Meses 1 a 6)

Las siguientes fases se ejecutan durante los **primeros 6 meses del programa**, en paralelo a las clases, con honorarios de **COP \$4,000,000 mensuales** (clases + consultoría de despliegues automatizados).

FASE 1: Preparación (Mes 1 — Semanas 1-2)

Actividad	Responsable
Crear organización GitHub y repositorios (1 por línea de producto)	DevOps QA + Instructor
Configurar branch protection rules y equipos	DevOps QA
Definir estrategia de workflows, secretos y modelo de runners (hosted vs self-hosted)	QA Lead + Instructor

FASE 2: Workflows y Configuración (Meses 1-2 — Semanas 3-8)

Actividad	Responsable
Crear workflows de build + deploy por línea de producto (Dev / QA / Prod con aprobaciones)	QA Lead + Instructor
Crear workflows de pruebas automatizadas por línea (JUnit, Karate, Playwright)	QA Lead + Instructor
Configurar secretos (credenciales BD, AS400, APIs, ambientes)	DevOps QA
Docker images para builds y suites de pruebas	DevOps + Instructor
Configurar GitHub Environments con aprobadores para Producción	DevOps QA
Configurar matriz de tests/deployes paralelos	QA Lead
Integración Slack + GitHub Pages para reportes y notificaciones de despliegue	DevOps QA

FASE 3: Pruebas y Estabilización (Meses 3-4)

Actividad	Responsable
Test y estabilización workflows de build + deploy para las 7 líneas de producto	DevOps QA + Instructor
Validación de despliegues automáticos a Dev con rollback automático	DevOps QA + Instructor
Validación de despliegues a QA con gates de calidad (tests verdes obligatorios)	QA Lead + Instructor
Validación de despliegues a Producción con flujo de aprobación GitHub Environments	QA Lead + DevOps + Instructor
Test y estabilización workflows JUnit (AS400 + APIs)	QA Engineers + Instructor
Test y estabilización workflows Karate	QA Engineers
Test y estabilización workflows Playwright (Web/Móvil)	QA Engineers
Optimizar timeouts, paralelismo y matrices (ajuste de costos de runners)	DevOps QA + Instructor
Configurar reportes en GitHub Pages y notificaciones de estado de despliegue	QA Lead

FASE 4: Mantenimiento y Transferencia (Meses 5-6)

Actividad	Frecuencia	Responsable
Monitoreo de runs fallidos	Diario	QA Team
Optimización de scripts y workflows	Semanal	QA Lead + Instructor
Actualización de dependencias	Mensual	DevOps
Transferencia de conocimiento y autonomía al equipo	Mensual	Instructor

Continuidad opcional (Meses 18 a 24)

- Endurecimiento de seguridad y gobierno CI/CD.
- Optimización avanzada de costos de runners/minutos.
- Escalabilidad multi-repositorio y estandarización corporativa.

⚠ Riesgos de Infraestructura

Riesgo	Probabilidad	Impacto	Mitigación
Tests muy lentos (>30 min)	Media	Alto	Paralelizar, usar matriz
Falsos positivos en tests	Alta	Medio	Usar waits inteligentes, fixtures estables
Runners no disponibles	Baja	Medio	Usar GitHub runners de backup
Credenciales expuestas	Muy Baja	Crítico	Usar GitHub Secrets, auditar logs

🔗 Recursos de Referencia

- [GitHub Actions Docs](#)
- [GitHub Actions Pricing](#)
- [JUnit 5 — junit.org](#)
- [Karate — karatelabs.io](#)
- [Playwright Java — playwright.dev/java](#)
- [Serenity BDD — serenity-bdd.info](#)

¿Dudas o ajustes

- ✉ alejoved@email.com
- 📞 +57 311 339 5602
- 💼 LinkedIn: [URL](#)

Última actualización: 21 de julio de 2026